day 26 prep

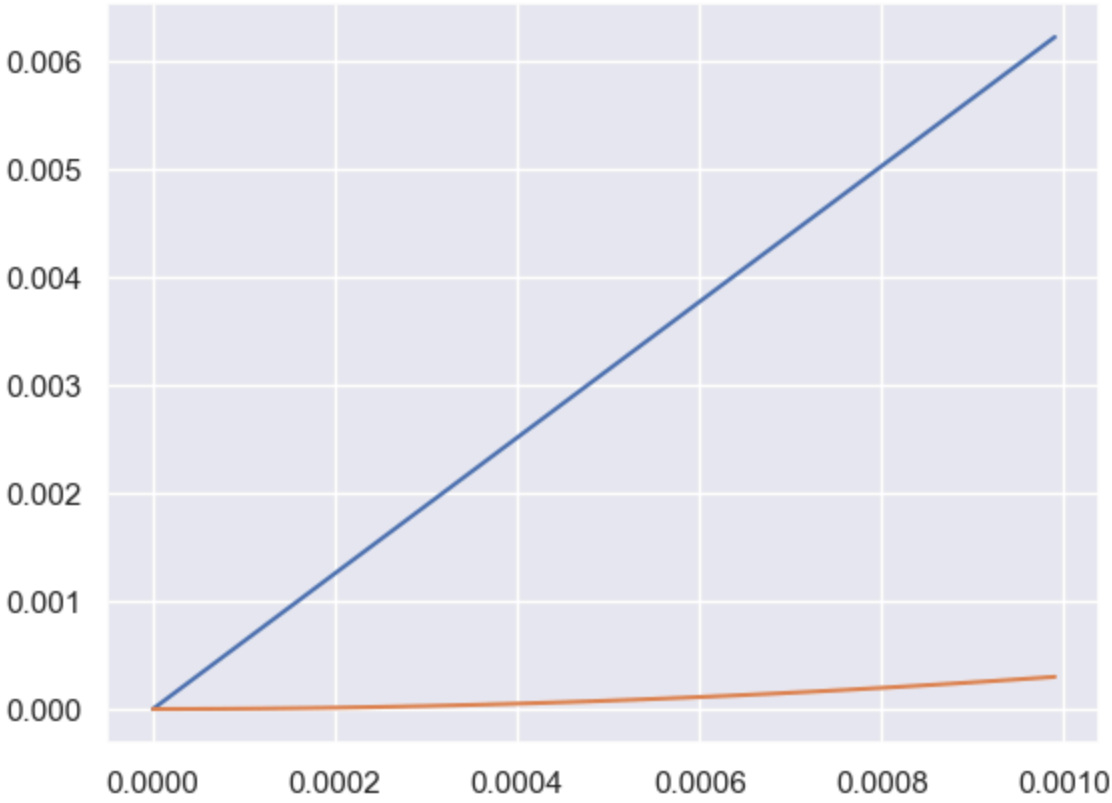
In [4]:
```
%matplotlib ipympl

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme()
```

Now do the above plot, but use a Sin wave, and rather than adjust RC, instead adjust the frequency of the wave. See how the low pass filter lets low frequencies go but stifles higher frequencies. But I haven't finished this yet....

In [18]:
```
# Runge-Kutta 4nd order

def vin(t):
    f = 1
    return(np.sin(2*np.pi*f*t))

def f(vout,t):
    return(1/0.01*(vin(t)-vout))

# define boundary conditions

a = 0.0 # starting point
b = 1.0 # ending point
N = 100000 # number of points between a and b
dt = (b-a)/N

x = 0.0 # initial condition


tpoints = np.arange(a, b, dt)
xpoints = []

for t in tpoints:
    xpoints.append(x)
    k1 = dt*f(x,t)
    k2 = dt*f(x+0.5*k1,t+0.5*dt)
    k3 = dt*f(x+0.5*k2,t+0.5*dt)
    k4 = dt*f(x+k3, t+dt)
    x = x + (k1+2*k2+2*k3+k4)/6
```

In [19]:
```
fig2, ax2 = plt.subplots(figsize=(3,3))
ax2.plot(tpoints[:100], vin(tpoints[:100]))
ax2.plot(tpoints[:100], xpoints[:100])
```

Out[19]:
```
[<matplotlib.lines.Line2D at 0x77505e390da0>]
```

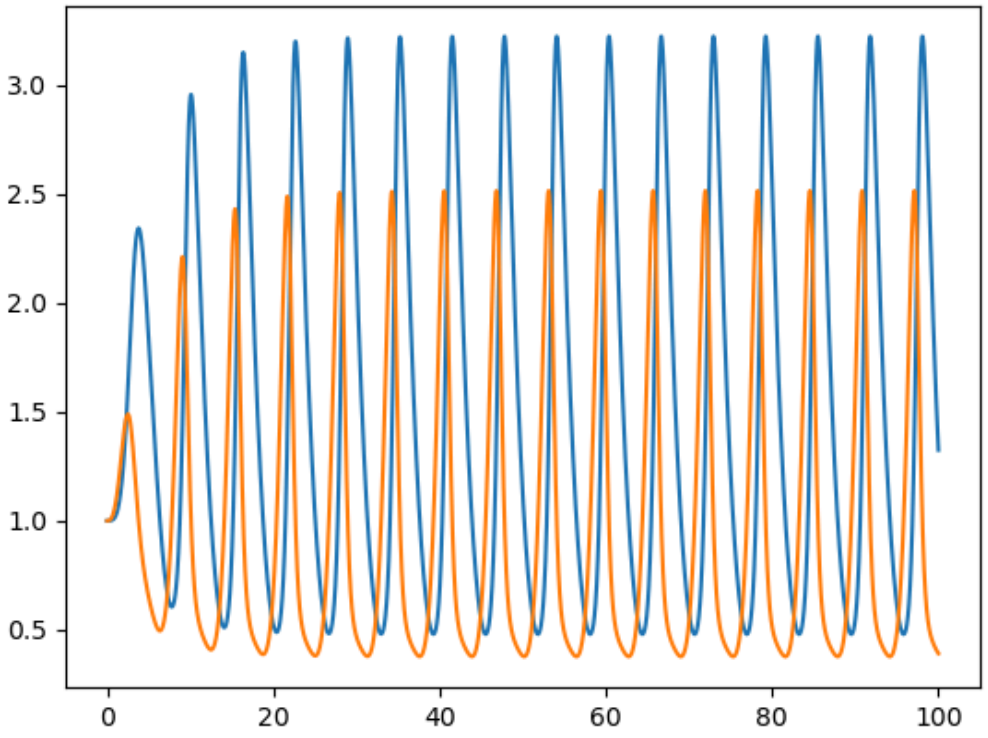


In [33]:
```python
# define the differential equation
def f(r, t): # don't forget r is a vector!
    x = r[0]
    y = r[1]
    w = 0.5
    fx = x*y-x  # this is the dx/dt equation!
    fy = y - x*y + np.sin(w*t)**2  # this is the dy/dt equation!
    return(np.array([fx, fy]))

# define the boundary condition
a = 0.0
b = 100.0
N = 10000
dt = (b-a)/N

# Runge-Kutta 4th order

r = np.array([1.0, 1.0]) # initial condition

tpoints = np.arange(a, b, dt)

xpoints = []
ypoints = []

for i in tpoints:
    xpoints.append(r[0])
    ypoints.append(r[1])
    k1 = dt*f(r, i)
    k2 = dt*f(r+1/2*k1, i+1/2*dt)
    k3 = dt*f(r+1/2*k2, i+1/2*dt)
```

```
    k4 = dt*f(r+k3, i+dt)
    r = r + (k1+2*k2+2*k3+k4)/6

fig, ax =  plt.subplots()
ax.plot(tpoints, xpoints)
ax.plot(tpoints, ypoints)
```

Out[33]: [<matplotlib.lines.Line2D at 0x773d59633500>]

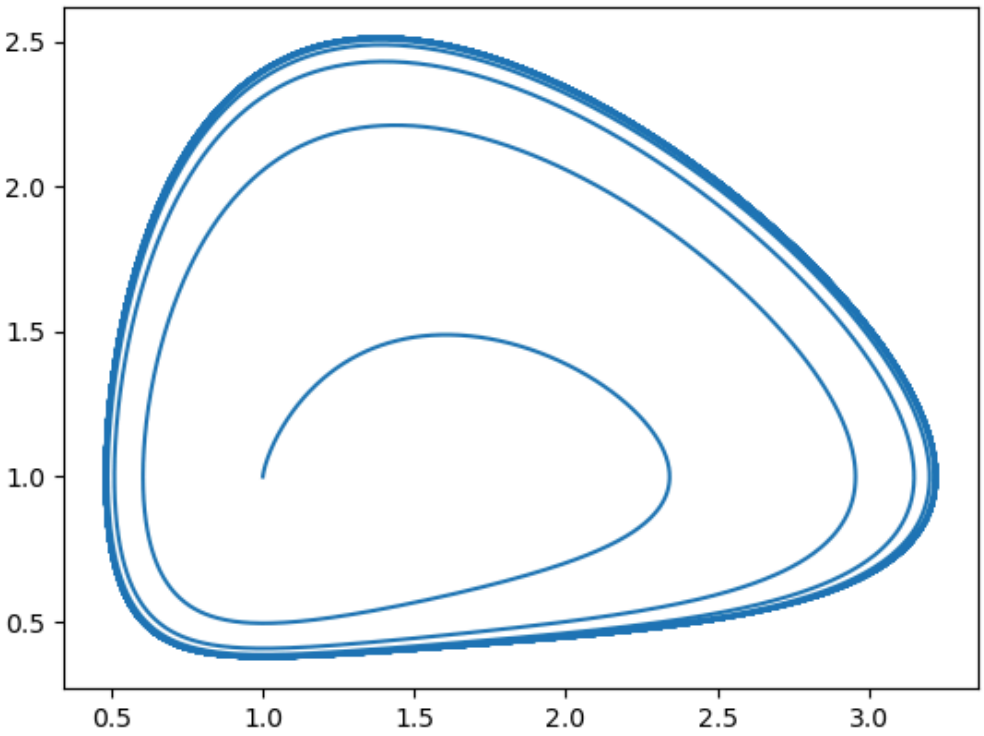
Figure



In [34]:
```
figt, axt = plt.subplots()
axt.plot(xpoints,ypoints)
```

Out[34]: [<matplotlib.lines.Line2D at 0x773d597d3710>]

Figure



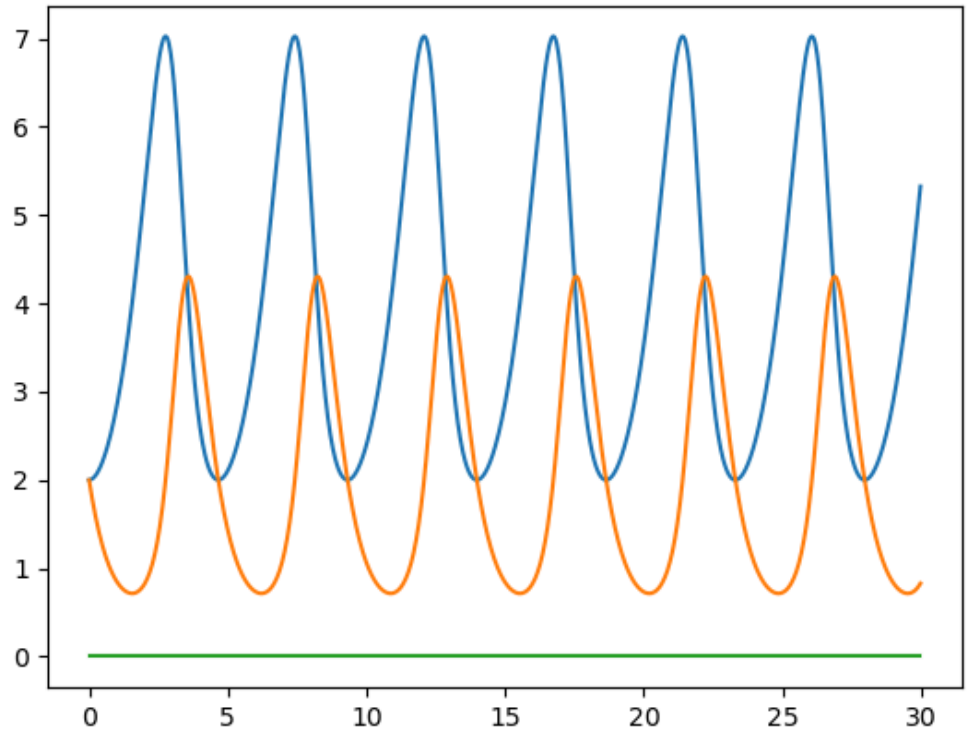
In [8]:

```
# define the differential equation
def f(r, t): # don't forget r is a vector!
    x = r[0]
    y = r[1]
    z = r[2]
    a = 1
    b = 0.5
    g = 0.5
    d = 2
    fx = a*x - b*x*y # this is the dx/dt equation!
    fy = g*x*y - d*y  # this is the dy/dt equation!
    fz = 0 # this is the dz/dt equation
    return(np.array([fx, fy, fz]))

# define the boundary condition
a = 0.0
b = 30.0
N = 1000
dt = (b-a)/N

# Runge-Kutta 4th order

r = np.array([2.0, 2.0, 0.0]) # initial condition

tpoints = np.arange(a, b, dt)

xpoints = []
```

```
ypoints = []
zpoints = []

for i in tpoints:
    xpoints.append(r[0])
    ypoints.append(r[1])
    zpoints.append(r[2])
    k1 = dt*f(r, i)
    k2 = dt*f(r+1/2*k1, i+1/2*dt)
    k3 = dt*f(r+1/2*k2, i+1/2*dt)
    k4 = dt*f(r+k3, i+dt)
    r = r + (k1+2*k2+2*k3+k4)/6

fig0, ax0 = plt.subplots()
ax0.plot(tpoints, xpoints)
ax0.plot(tpoints, ypoints)
ax0.plot(tpoints, zpoints)
```

Out[8]: [<matplotlib.lines.Line2D at 0x773d5a6a3410>]

Figure

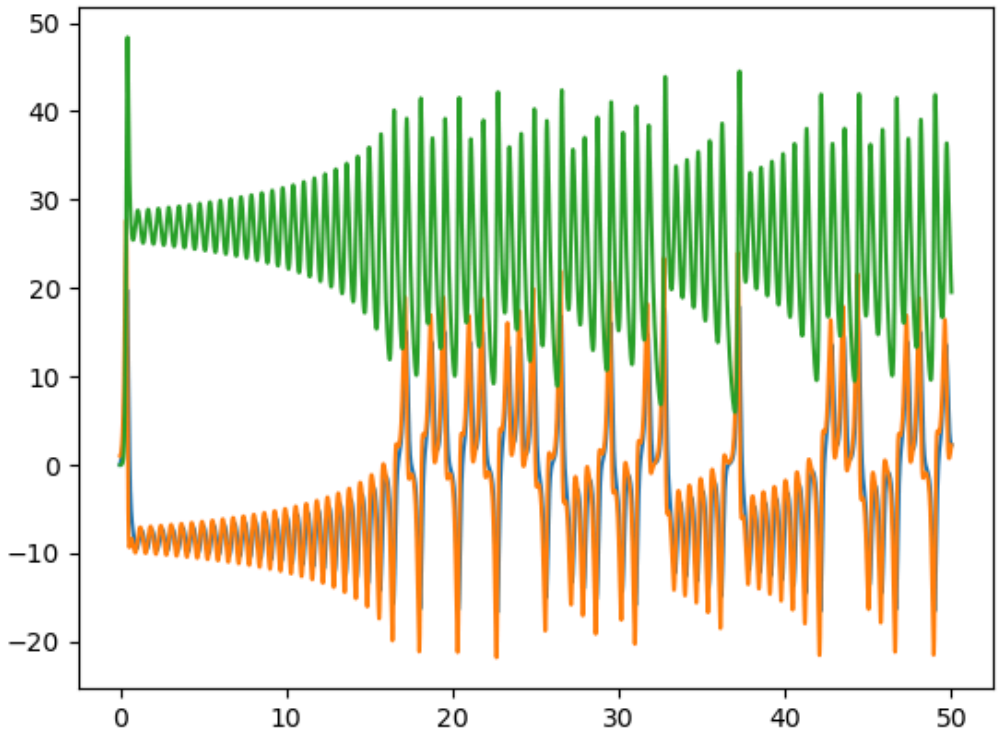


In [41]:
```
# define the differential equation
def f(r, t): # don't forget r is a vector!
    x = r[0]
    y = r[1]
    z = r[2]
    s = 10
    r = 28
    b = 8/3.
```

```
    fx = s*(y-x) # this is the dx/dt equation!
    fy = r*x - y - x*z  # this is the dy/dt equation!
    fz = x*y - b*z # this is the dz/dt equation
    return(np.array([fx, fy, fz]))

# define the boundary condition
a = 0.0
b = 50.0
N = 10000
dt = (b-a)/N

# Runge-Kutta 4th order

r = np.array([0.0, 1.0, 0.0]) # initial condition

tpoints = np.arange(a, b, dt)

xpoints = []
ypoints = []
zpoints = []

for i in tpoints:
    xpoints.append(r[0])
    ypoints.append(r[1])
    zpoints.append(r[2])
    k1 = dt*f(r, i)
    k2 = dt*f(r+1/2*k1, i+1/2*dt)
    k3 = dt*f(r+1/2*k2, i+1/2*dt)
    k4 = dt*f(r+k3, i+dt)
    r = r + (k1+2*k2+2*k3+k4)/6

fig1, ax1 = plt.subplots()
ax1.plot(tpoints, xpoints)
ax1.plot(tpoints, ypoints)
ax1.plot(tpoints, zpoints)
```

Out[41]: [<matplotlib.lines.Line2D at 0x773d5161c530>]

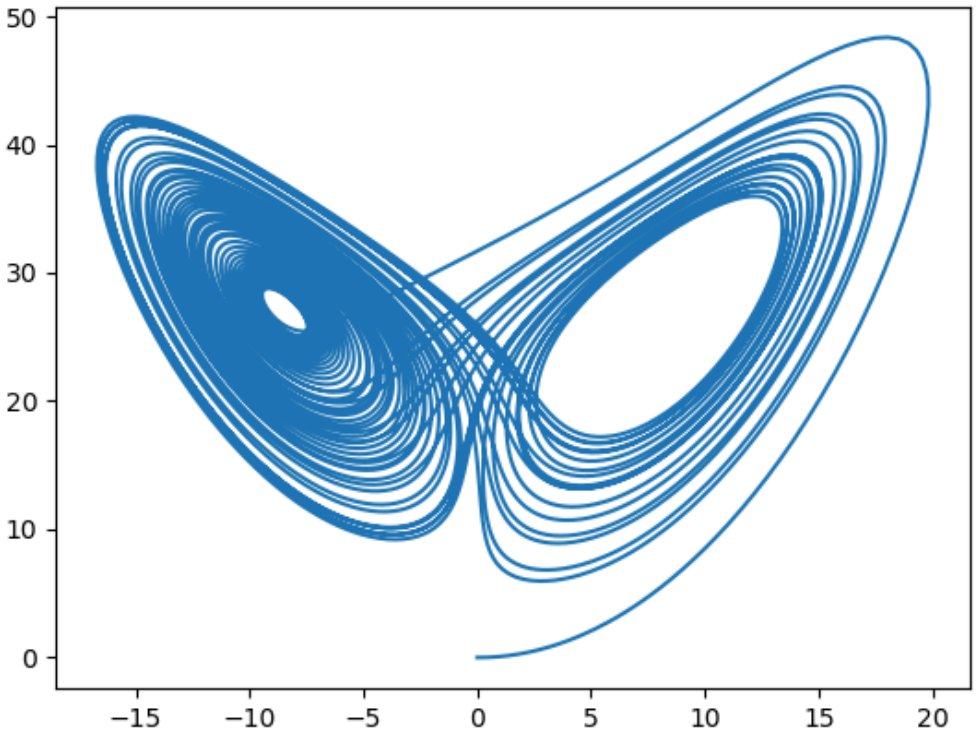
Figure



In [42]:
```
fig2, ax2 = plt.subplots()
ax2.plot(xpoints, zpoints)
```

Out[42]: [<matplotlib.lines.Line2D at 0x773d5104f710>]

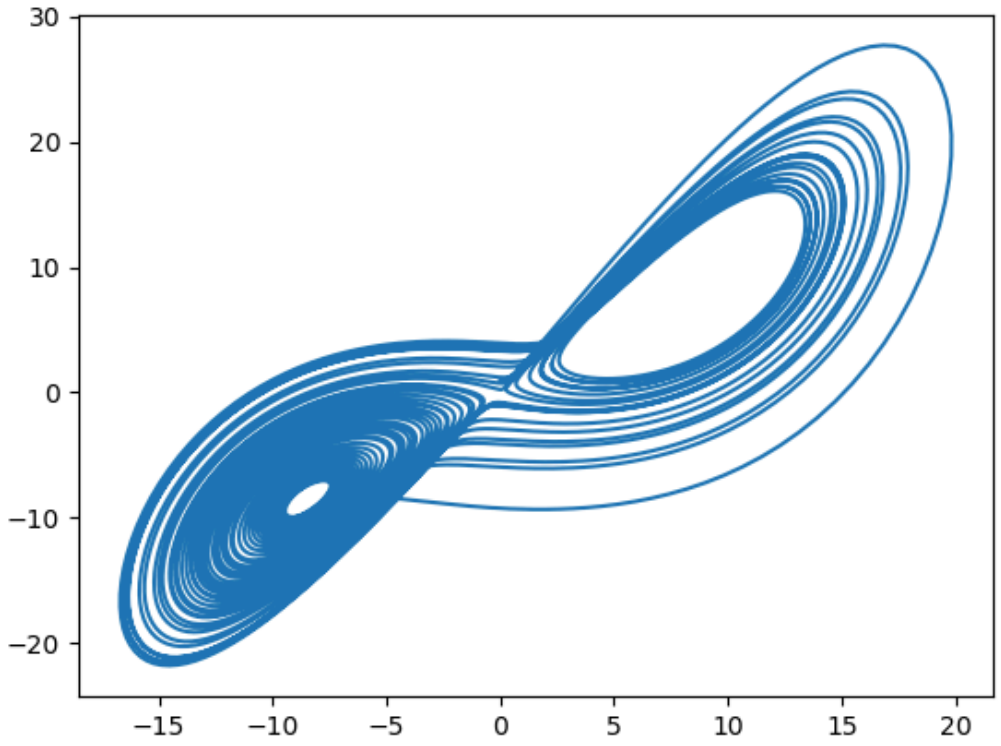
Figure



In [14]:
```
fig3, ax3 = plt.subplots()
ax3.plot(xpoints, ypoints)
```

Out[14]:
```
[<matplotlib.lines.Line2D at 0x773d5a0d3aa0>]
```

Figure



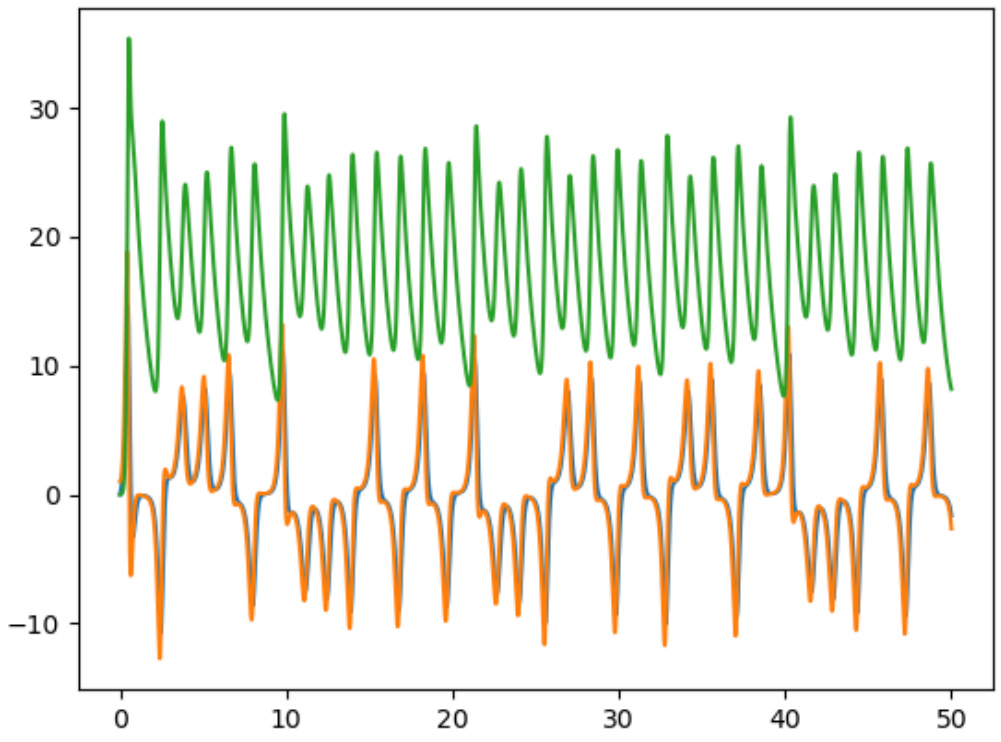
In [ ]:

In [22]:
```
# define the differential equation
def f(r, t): # don't forget r is a vector!
    x = r[0]
    y = r[1]
    z = r[2]
    s = 10
    r = 20
    b = 1
    fx = s*(y-x) # this is the dx/dt equation!
    fy = r*x - y - x*z  # this is the dy/dt equation!
    fz = x*y - b*z # this is the dz/dt equation
    return(np.array([fx, fy, fz]))

# define the boundary condition
a = 0.0
b = 50.0
N = 10000
dt = (b-a)/N

# Runge-Kutta 4th order

r = np.array([0.0, 1.0, 0.0]) # initial condition

tpoints = np.arange(a, b, dt)
```

```
xpoints = []
ypoints = []
zpoints = []

for i in tpoints:
    xpoints.append(r[0])
    ypoints.append(r[1])
    zpoints.append(r[2])
    k1 = dt*f(r, i)
    k2 = dt*f(r+1/2*k1, i+1/2*dt)
    k3 = dt*f(r+1/2*k2, i+1/2*dt)
    k4 = dt*f(r+k3, i+dt)
    r = r + (k1+2*k2+2*k3+k4)/6

fig4, ax4 = plt.subplots()
ax4.plot(tpoints, xpoints)
ax4.plot(tpoints, ypoints)
ax4.plot(tpoints, zpoints)
```

Out[22]: `[<matplotlib.lines.Line2D at 0x773d59b627e0>]`

Figure

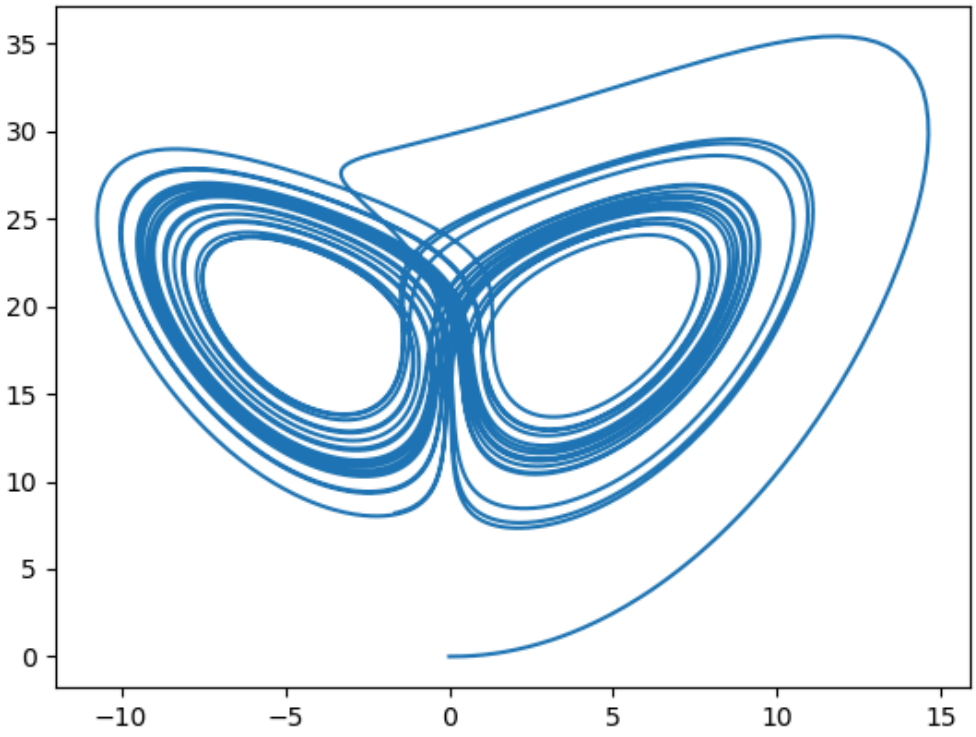


In [25]:
```
fig5, ax5 = plt.subplots()
ax5.plot(xpoints, zpoints)
```

```
/tmp/ipykernel_565587/4255734765.py:1: RuntimeWarning: More than 20 figures
have been opened. Figures created through the pyplot interface (`matplotlib.
pyplot.figure`) are retained until explicitly closed and may consume too muc
h memory. (To control this warning, see the rcParam `figure.max_open_warning
`). Consider using `matplotlib.pyplot.close()`.
  fig5, ax5 = plt.subplots()
```

Out[25]: `[<matplotlib.lines.Line2D at 0x773d596616d0>]`

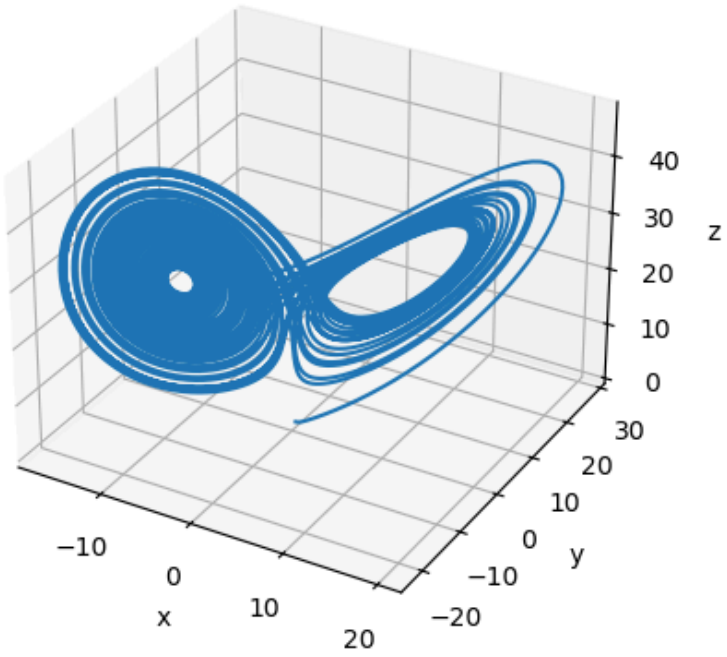
Figure



In [45]:
```
fig6 = plt.figure()
ax6 = fig6.add_subplot(projection='3d')
ax6.plot(xpoints, ypoints, zpoints)
ax6.set_xlabel('x')
ax6.set_ylabel('y')
ax6.set_zlabel('z')
```

Out[45]: `Text(0.5, 0, 'z')`

Figure



In [ ]: